Workplan for Teaching Assistants

Man power planning, i.e. deciding which worker/employee does which job, is a classic Operations Research problem. The work plan for teaching assistants problem involves assigning TAs to certain time slots, such that enough TAs are available to help the students during the different time slots.

Problem

- Minimize the TA inconvenience while covering the TA demand.

Sets

- Teaching Assistants: $ta \in TeachingAssistants = \{AL, FR, JE, MI\}$
- Time slots: $p \in Periods = \{"9-10", "10-11", "11-12", "12-13", "13-14", "14-15", "15-16", "16-17"\}$
- Days: $d \in Days = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Parameters

- $Demand_{p,d}$: The number of TAs necessary in time slot p on day d
- $Inconvenience^{ta}_{p,d}$: The inconvenience value for ta to work in time slot p on day d

Decision variables

- Should TA ta work in time slot p on day d: $x^{ta}_{p,d} \in \{0, 1\}$.

Model

Objective:

- Minimize the total TA inconvenience:

$$\sum_{ta,p,d} Inconvenience_{p,d}^{ta} \cdot x_{p,d}^{ta}$$

Constraints:

- Ensure that the TA demand is satisfied:

$$\sum_{ta} x_{p,d}^{ta} \geq Demand_{p,d} \; \forall \; p,d$$

- Ensure that each TA is working exactly the correct no. of hours:

$$\sum_{p,d} x_{p,d}^{ta} = 52 \; \forall \; ta$$

The model is actually rather simple.

The full model in Julia/JuMP, available with the name

`WorkplanTeachingAssistant1.jl`

from the book web-site, is given below:

```
#*****************************************************************************
# Workplan Teaching Assistant Assignment 1 question,
# "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
#*****************************************************************************

#*****************************************************************************
# PARAMETERS
include("WorkplanData.jl")
#*****************************************************************************

#*****************************************************************************
# Model
taworkplan = Model(HiGHS.Optimizer)

# 1 if ta is working in time period p on day d
@variable(taworkplan, x[ta=1:TA,p=1:P,d=1:D],Bin)

# Minimize summed Inconvenience
@objective(taworkplan, Min,
```

```
            sum( Inconvenience[ta,p,d]*x[ta,p,d]
                 for ta=1:TA,p=1:P,d=1:D))

# satisfy demand for TAs for all periods, p, and days, d:
@constraint(taworkplan, [p=1:P,d=1:D],
             sum( x[ta,p,d] for ta=1:TA) >= Demand[p,d])

# each TA should work exactly 52 hours
@constraint(taworkplan, [ta=1:TA],
             sum( x[ta,p,d] for p=1:P,d=1:D) == 52)
#*****************************************************************************


#*****************************************************************************
# solve
optimize!(taworkplan)
println("Termination status: $(termination_status(taworkplan))")
#*****************************************************************************


#*****************************************************************************
println("-------------------------------------");
if termination_status(taworkplan) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(taworkplan))")

    for ta=1:TA
        println("")
        println("")
        print("TA: $(TA_acronyms[ta])")
        for d=1:D
            print("\t$(d)")
        end
        println("")
        for p=1:P
            print("$(Periods[p])\t")
            for d=1:D
                if round.(Int8,value(x[ta,p,d]))==1
                    print("*\t")
                else
                    print("\t")
                end
            end
            println("")
        end
    end
```

```
else
    println("  No solution")
end
println("---------------------------------------");
#**************************************************************************
```

Comments to the Julia/JuMP program:

- Notice the printout method in the bottom of the program. Implemented to give a simple, easy to read, plan for each TA.

Problem 2

Question 2 in this assignment is about ensuring that the TAs can only work consecutive time slots on a day and at least 2 hours when they work. One new binary variable $y_{p,d}^{ta}$ is added, and it is 1 if TA ta works their first time slot p on day d. Furthermore *three* constraints ensure that the requirements are satisfied.

Decision variables

- Should TA ta start the work on day d in time slot p: $y_{p,d}^{ta} \in \{0, 1\}$.

Constraints:

- Ensure that the TA can at most start to work once a day:

$$\sum_{p} y_{p,d}^{ta} \leq 1 \ \forall \ ta, d$$

- Ensure that a TA can only work in time period p if they worked in the previous time period or they start working this time period (note the use of a conditional statement to avoid addressing the time period before the first time period since it does not exist):

$$x_{p,d}^{ta} \leq x_{p-1,d}^{ta} | (p > 1) + y_{p,d}^{ta} \ \forall \ ta, p, d$$

- If a TA works on a day, then the TA has to work at least 2 time slots that day:

$$\sum_{p} x_{p,d}^{ta} \geq 2 \cdot \sum_{p} y_{p,d}^{ta} \ \forall \ ta, d$$

The full model in Julia/JuMP, available with the name

`WorkplanTeachingAssistant2.jl`

from the book web-site, is given below:

```
#*****************************************************************************
# Workplan Teaching Assistant Assignment question 2,
# "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
#*****************************************************************************

#*****************************************************************************
# PARAMETERS
include("WorkplanData.jl")
#*****************************************************************************

#*****************************************************************************
# Model
taworkplan = Model(HiGHS.Optimizer)

# 1 if ta is working oin time period p on day d
@variable(taworkplan, x[ta=1:TA,p=1:P,d=1:D],Bin)

# 1 if ta STARTS working in period p on day d
@variable(taworkplan, y[ta=1:TA,p=1:P,d=1:D],Bin)

# Minimize summed Inconvenience
@objective(taworkplan, Min,
           sum( Inconvenience[ta,p,d]*x[ta,p,d]
                for ta=1:TA,p=1:P,d=1:D))

# satisfy demand for TAs
@constraint(taworkplan, [p=1:P,d=1:D],
            sum( x[ta,p,d] for ta=1:TA) >= Demand[p,d])

# each TA should work exactly 52 hours
@constraint(taworkplan, [ta=1:TA],
            sum( x[ta,p,d] for p=1:P,d=1:D) == 52)

# only start working once a day
@constraint(taworkplan, [ta=1:TA,d=1:D],
            sum( y[ta,p,d] for p=1:P) <= 1 )

# require connected work plans
@constraint(taworkplan, [ta=1:TA,d=1:D,p=1:P],
            x[ta,p,d] <= (p>1 ? x[ta,p-1,d] : 0) + y[ta,p,d])
```

```
# at least 2 hours of work pr. day if working that day
@constraint(taworkplan, [ta=1:TA,d=1:D],
            sum( x[ta,p,d] for p=1:P) >= 2*sum( y[ta,p,d] for p=1:P) )

#**************************************************************************


#**************************************************************************
# solve
optimize!(taworkplan)
println("Termination status: $(termination_status(taworkplan))")
#**************************************************************************


#**************************************************************************
println("---------------------------------------");
if termination_status(taworkplan) == MOI.OPTIMAL
    println("RESULTS:")
    println("objective = $(objective_value(taworkplan))")

    for ta=1:TA
        println("")
        println("")
        print("TA: $(TA_acronyms[ta])")
        for d=1:D
            print("\t$(d)")
        end
        println("")
        for p=1:P
            print("$(Periods[p])\t")
            for d=1:D
                if round.(Int8,value(x[ta,p,d]))==1
                    print("*\t")
                else
                    print("\t")
                end
            end
            println("")
        end
    end
else
    println("  No solution")
end
println("---------------------------------------");
#**************************************************************************
```

Comments to the Julia/JuMP program:

- The printout makes it easy to see that the new constraints are satisfied. But the added constraints makes the optimal inconvenience value worse.